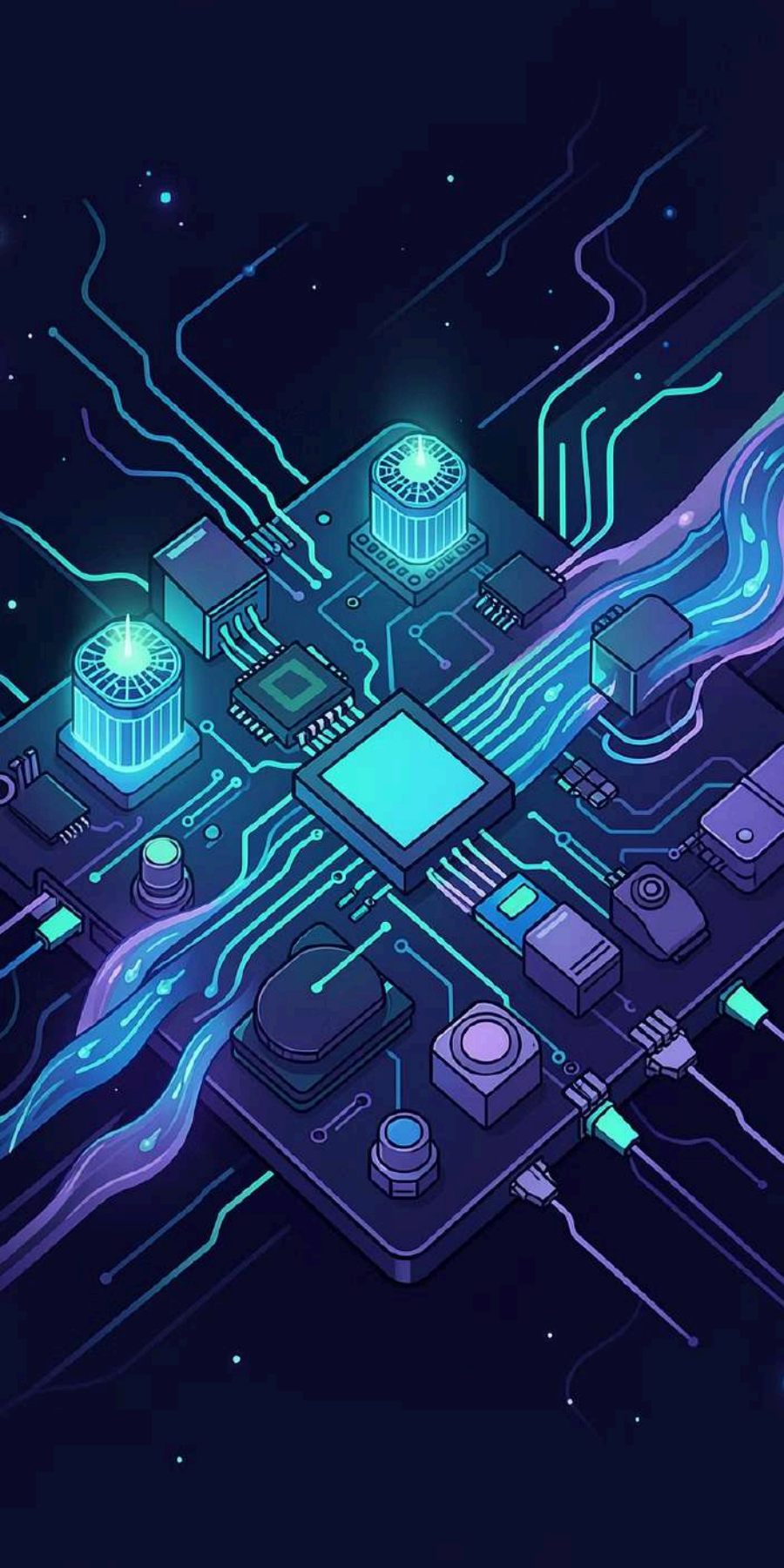




Smart Irrigation Controller with AI-Powered Pump Fault Detection

An Edge IoT System Using M5Stack Core2, MicroPython, PyTorch & Flask

🔧 EMBEDDED SYSTEMS

🧠 MACHINE LEARNING

📶 IOT

</> MICROPYTHON

Project by Aryan Gupta, Bhavika Pandya, Daivik Gupta, and Shourya Goyal

The Problem: Irrigation Pumps Fail Silently



→ Silent Failures

Agricultural and domestic irrigation pumps are prone to faults — dry runs, bearing wear, cavitation, and motor stress

→ Undetected Until Breakdown

Failures go undetected until total breakdown, causing crop damage and costly repairs

→ Expensive or Manual Solutions

Existing solutions rely on manual inspection or expensive industrial sensors

→ No Affordable Automation

No affordable, automated real-time fault detection exists for small-scale users

→ Costly Consequences

Water waste and pump burnout cost farmers and homeowners thousands annually

Up to 60% of pump failures can be predicted from acoustic signatures before breakdown



Our Solution: Listen, Classify, Act — In Real Time



Listen

- M5Stack Core2 (ESP32) records live pump audio via its internal PDM microphone
- 16kHz PCM audio captured directly using the M5 hardware library
- Audio stored on SD card for offline analysis



Classify

- Raw audio streamed to a laptop-hosted Flask server over WiFi
- PyTorch MLP model extracts 13 MFCC features and classifies: Normal vs. Fault
- Inference takes < 500ms per audio segment



Act

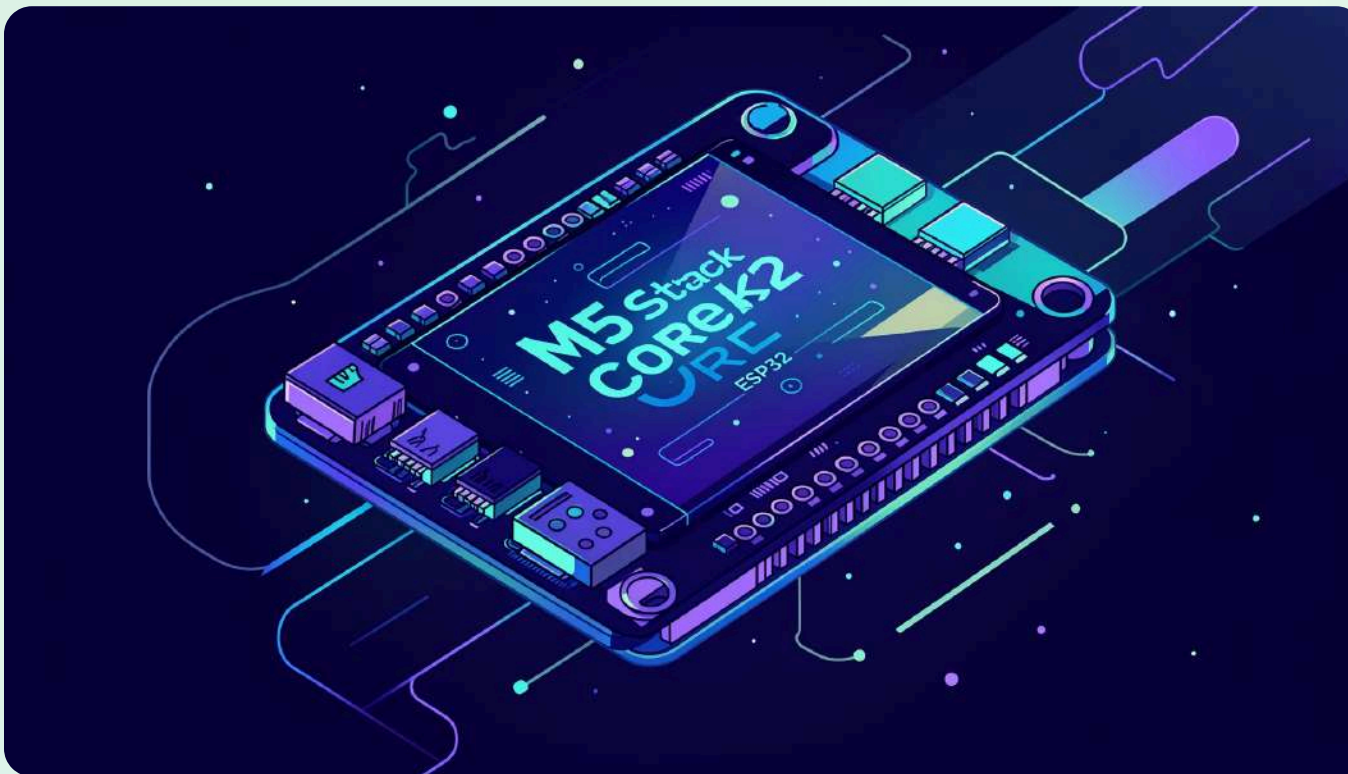
- If fault is detected, the relay shutter is triggered immediately
- Pump is auto-shutdown with status logged as "AUTO-SHUTDOWN (FAULT)"
- Web UI notifies operator with last action status in real time

System Architecture: End-to-End Data Flow



The architecture flows from edge capture on the M5Stack Core2 through WiFi to a Flask inference server running PyTorch, with fault responses triggering immediate relay shutdown — all under 1 second.

Hardware: M5Stack Core2 as the Edge Node



Hardware Specs

- **MCU:** ESP32-D0WDQ6-V3 dual-core 240MHz
- **Internal microphone:** PDM type, hardware-filtered via M5 library
- **MicroSD slot:** SPI bus (SCK=18, MOSI=23, MISO=38, CS=4)
- **Relay module:** GPIO Pin 33, active HIGH
- **WiFi:** 802.11 b/g/n, 2.4GHz (native ESP32)
- **Power:** USB-C or LiPo battery

Firmware: Modular MicroPython on ESP32

main.py

Entry point — Initializes SD, mic, controller, WiFi and launches HTTP web server

config.py

All tunable parameters in one place — WiFi SSID/password, GPIO pins, timing constants, server IP

audio_manager.py

- `init_sd()`: Mounts SD card via SPI
- `init_mic()`: Configures M5.Mic at 16kHz
- `record_audio()`: Buffers PCM in 1024-byte DMA chunks
- `send_for_analysis()`: HTTP POST to Flask server

relay_control.py

Wraps GPIO Pin 33 — `relay_on()`, `relay_off()`, `relay_toggle()`, `relay_state()`

controller.py

- State machine: OFF / ON / TIMED modes
- `update_controller()`: Runs timed shutdown + auto ML monitoring
- Fault detection triggers `relay_off()` + logs "AUTO-SHUTDOWN (FAULT)"

web_server.py

- MicroPython socket server on port 8080
- Serves HTML UI; handles `/on`, `/off`, `/toggle`, `/run`, `/record`, `/download`, `/clear_audio`
- File download in 1KB chunks to conserve RAM

Machine Learning: Training the Pump Fault Detector

1

Dataset

WAV files organized in dataset/normal/ and dataset/fault/ folders

2

Feature Extraction

`librosa.feature.mfcc(n_mfcc=13, sr=16000)` → mean across time axis → 13-dimensional vector per file

3

Train/Test Split

80/20 random split (sklearn)

4

Model Architecture

Feed-forward MLP — Linear(13→32) → ReLU → Dropout(0.2) → Linear(32→16) → ReLU → Linear(16→2)

5

Loss & Optimizer

CrossEntropyLoss | Optimizer: Adam (lr=0.001)

6

Training Config

Epochs: 50 | Batch Size: 8

7

Result

High Test Accuracy on 59 training samples

Inference Server: Real-Time Audio Classification

1

Receive

Receive raw PCM bytes via HTTP POST /analyze

3

Unpack PCM

Unpack 16-bit signed PCM: `struct.unpack("<{n}h", raw_bytes)`

5

Extract MFCCs(Mel-Frequency Cepstral Coefficients)

`librosa.feature.mfcc(y=audio_np, sr=16000, n_mfcc=13)`

7

Inference

`torch.no_grad() → model → torch.max → predicted class`

2

Align Bytes

Ensure even byte count: `raw_bytes[:len//2 * 2]`

4

Normalize

`audio_np / 32768.0 → float32 array`

6

Mean-Pool

`np.mean(mfccs.T, axis=0) → shape (13,)`

8

Return JSON

`jsonify({"status": "normal"})` or `jsonify({"status": "fault"})`

Step-by-Step Code Walkthrough

Web UI: Real-Time Control from Any Browser

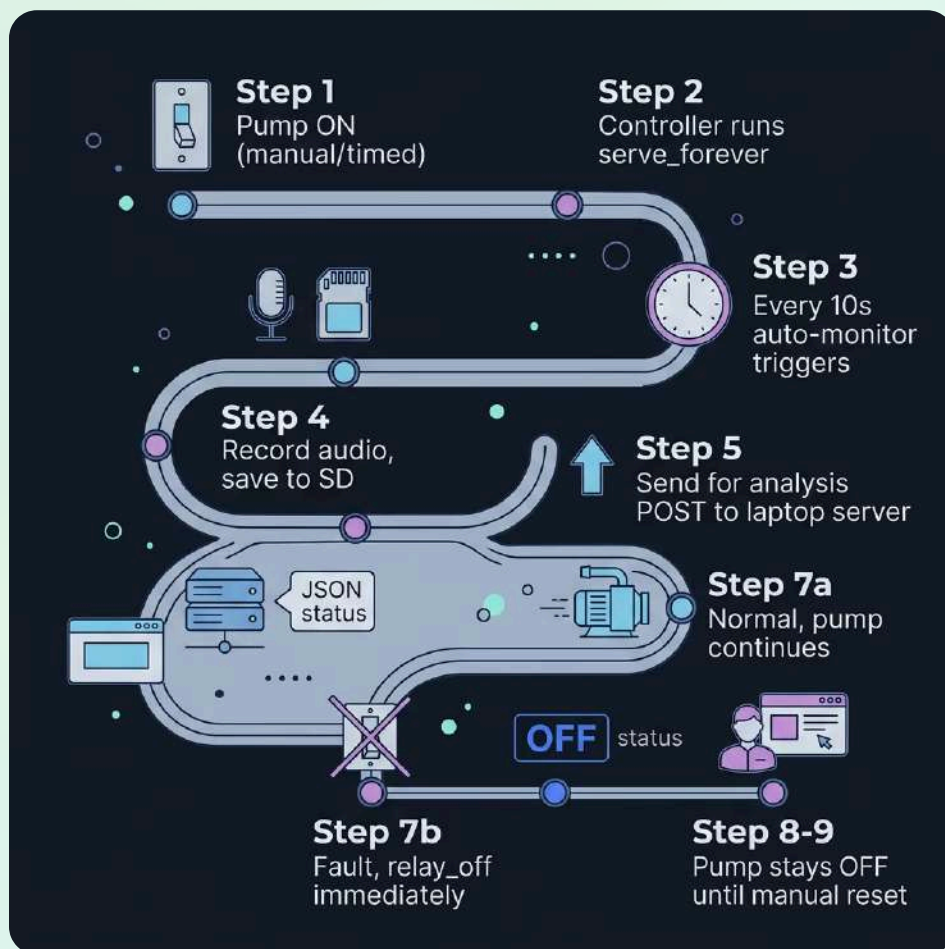
Features

- Hosted directly on ESP32 — no internet required, fully local
- Accessible via device IP at port 8080 on local WiFi
- Controls: ON / OFF / TOGGLE / RUN 2s / RUN 5s / RECORD 5s
- Status dashboard: *Current mode, relay raw state, seconds remaining, last action*
- SD card file browser: *lists all .raw recordings sorted newest-first*
- Per-file download links (*binary stream in 1KB RAM-safe chunks*)
- CLEAR ALL button with confirmation dialog
- Auto-redirect after record/clear actions (303 See Other)

Route Table

Endpoint	Action
GET /	Dashboard HTML
GET /on	Pump ON
GET /off	Pump OFF
GET /toggle	Toggle relay
GET /run? seconds=N	Timed run
GET /record	Trigger 5s recording
GET /download? file=X	Download .raw file
GET /clear_audio	Delete all recordings

Autonomous Operation: The Monitoring Loop



Key Design Highlight

"Zero human intervention required — from acoustic signature to relay shutdown in under 1 second"

Engineering Challenges & Design Decisions

Challenge	Solution
MicroPython has no audio DSP libraries	Used M5.Mic.record() with hardware PDM filter — outputs clean 16kHz PCM natively
ESP32 RAM too limited for ML inference	Offloaded model to laptop Flask server; ESP32 only records and sends raw bytes
SD card mount inconsistency across boots	Check <code>os.listdir("/")</code> first; fall back to manual SPI mount only if needed
Timed run + monitoring must coexist	Non-blocking <code>update_controller()</code> called in HTTP serve loop; uses <code>ticks_diff</code> for scheduling
Binary file download on MicroPython sockets	Chunked 1KB reads with <code>client.send()</code> — avoids loading full file into RAM
Model overfitting on small dataset	Dropout(0.2) in training; MFCC mean pooling gives stable, time-invariant features
Preventing circular imports in MicroPython	<code>audio_manager</code> imports done inline inside <code>update_controller()</code> function body

Results & What's Next



Results Achieved

- Fully functional IoT pump controller deployed on M5Stack Core2
- Real-time fault detection latency: < 1 second (record → classify → shutdown)
- ML model: high test accuracy on 59 labeled audio samples
- Web UI accessible on local WiFi with full control and file management
- Auto-shutdown loop tested and operational with configurable intervals



Future Scope

- Deploy TensorFlow Lite / ONNX model directly on ESP32 (eliminate server dependency)
- Expand dataset with diverse fault types: cavitation, dry-run, bearing failure
- Add cloud push notifications (Telegram bot, email alerts)
- Add soil moisture sensor for closed-loop irrigation scheduling
- Integrate with solar power for fully off-grid deployment
- Multi-pump support with centralized dashboard



Why This Matters

- Affordable alternative to industrial vibration/acoustic fault sensors (costs < ₹5,000 total)
- Applicable to agriculture, water treatment, HVAC, and industrial pump monitoring
- Demonstrates edge-AI viability on consumer microcontrollers

A microcontroller. A trained neural network. Zero manual intervention.

Tech Stack Summary

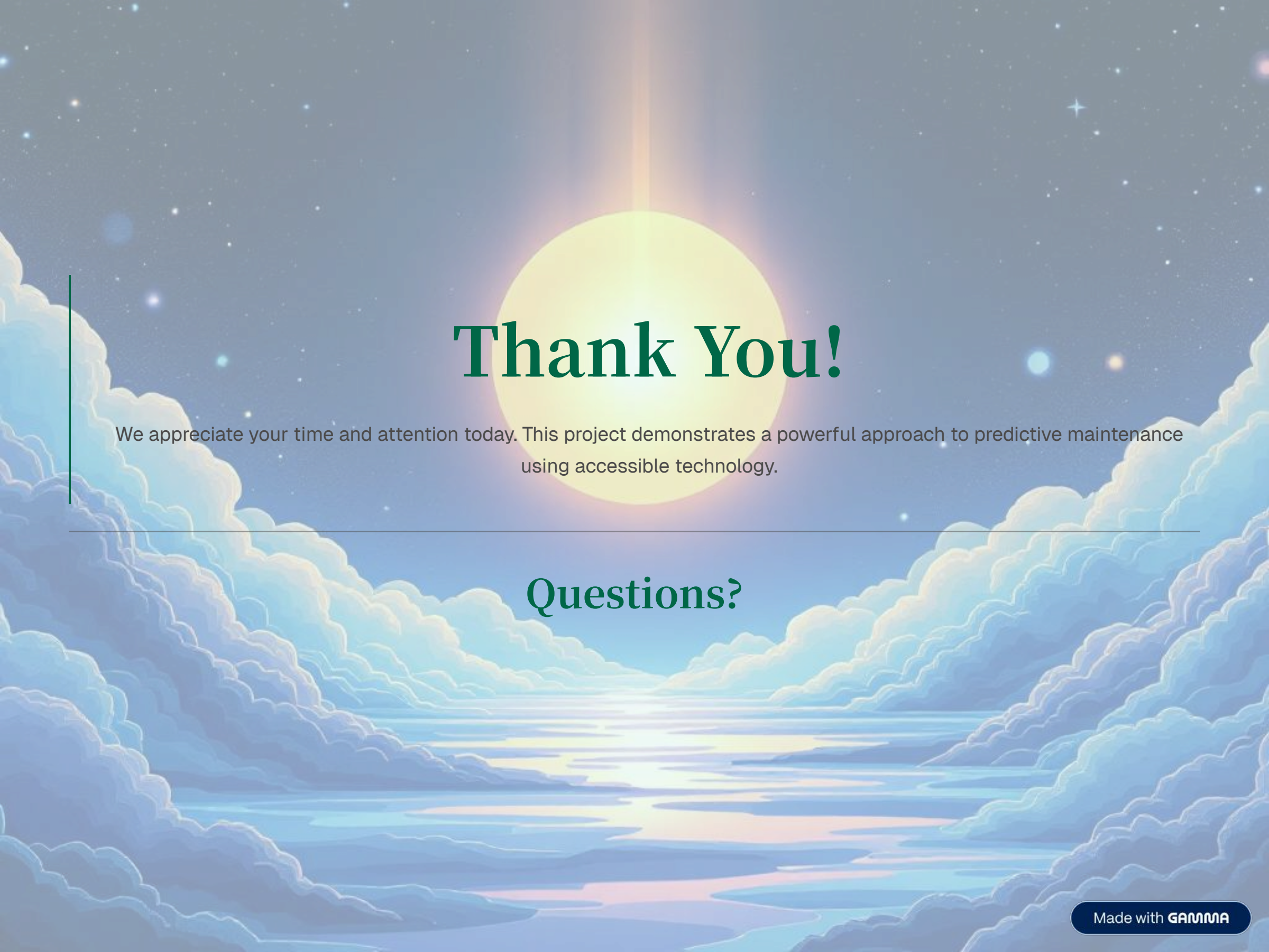
This project leverages a diverse set of hardware and software technologies to deliver intelligent irrigation control.

Layer	Technology
Microcontroller	M5Stack Core2 (ESP32, 240MHz)
Firmware Language	MicroPython
Audio Capture	M5 Unified Library (PDM → PCM)
Storage	MicroSD via SPI
Networking	ESP32 WiFi + MicroPython Socket
ML Feature Extraction	librosa (MFCC, 13 coefficients)
ML Framework	PyTorch (MLP, CrossEntropyLoss)
Inference Server	Python Flask (REST API)
Training Environment	Jupyter Notebook

File/Module Map

A high-level overview of the project's file and module organization, detailing the purpose of each component.

File/Module	Description
project/	Root directory of the project.
├── main.py	The primary boot entry point for the MicroPython application on the ESP32.
├── config.py	Contains all constants, configuration parameters, and pin definitions for the hardware.
├── boot.py	Handles the M5Stack UIFlow2 boot process, including initial setup and WiFi connection.
├── controller.py	Implements the pump state machine logic and manages the autonomous monitoring loop.
├── relay_control.py	Provides an abstraction layer for controlling the GPIO-connected pump relay.
├── audio_manager.py	Manages microphone initialization, audio recording, SD card storage, and HTTP upload of audio data.
├── web_server.py	Hosts the HTTP server, defines routes, and serves the web-based user interface.
├── wifi_manager.py	Handles WiFi connection establishment and status management for the ESP32.
├── inference_server.py	The Flask-based machine learning inference server, designed to run on a connected laptop.
├── ml_model.ipynb	Jupyter Notebook containing the code for training the pump fault detection model.
└── pump_fault_model.pth	The trained PyTorch model weights used by the inference server for classification.



Thank You!

We appreciate your time and attention today. This project demonstrates a powerful approach to predictive maintenance using accessible technology.

Questions?